



Computer Science Department
Summer Internship Report
Second Year in Software Engineering
Presented by: **Yassine Ben Jemaa**

**Design and Implementation of a
Real-Time Anomaly Detection Platform
for Branch Teller Operations
using Deep Learning**

Supervisor: **Mr. Oussama Ben Cheikh**

Realized within:



Academic Year: 2025–2026

Acknowledgements

I would like to express my sincere gratitude to Amen Bank for offering me the opportunity to complete this internship within the Direction Centrale du Système d’Information. I would especially like to thank my supervisor, Mr. Oussama Ben Cheikh, for his guidance and for granting me the freedom to explore beyond the original project scope.

This experience allowed me to acquire practical skills in deep learning, streaming architectures, and MLOps, and to discover the challenges of building production-oriented systems in a real banking environment.

I would also like to thank the Faculté des Sciences de Tunis and the University of Tunis El Manar for the academic foundation that made this work possible.

Contents

Acknowledgements	i
List of Abbreviations	vii
General Introduction	1
1 Context and State of the Art	2
1.1 Host Institution	2
1.2 Branch Teller Operations	2
1.3 Regulatory Framework	2
1.4 Existing Detection Capabilities and Project Positioning	3
1.5 State of the Art	3
1.5.1 Rule-Based and Statistical Detection	3
1.5.2 Autoencoders	3
1.5.3 LSTM Networks	4
1.5.4 Score Fusion	4
1.5.5 Explainability with SHAP	4
1.5.6 Streaming Architectures	4
1.6 Problem Statement and Objectives	4
2 System Architecture and Design	6
2.1 System Overview	6
2.2 Streaming Topology	6
2.3 Hot/Cold Data Layer	6
2.4 Raw Event Schema	7
2.5 Dual-Actor Profiling	7
2.6 Enriched Event Design	8
2.6.1 The Profile Dominance Problem	8
2.6.2 Contrast Feature Redesign	8
2.7 Scoring Architecture	9
2.7.1 Dual-Model Rationale	9
2.7.2 Score Fusion	9
2.7.3 Cold-Start Scoring	9
2.7.4 Conditional SHAP	9
2.8 Decision Service	10
2.9 Software Architecture	10
3 Synthetic Data Engineering	12
3.1 Source Extraction Methodology	12

3.2	Source Extractions	12
3.3	Client Archetype Design	13
3.4	Generator Architecture	13
3.4.1	Branch-Linked Employee Assignment	13
3.5	Anomaly Scenario Design	14
3.6	Data Validation	14
3.6.1	Oracle Profile Shortcut	15
4	Model Training and Evaluation	16
4.1	Data Preparation	16
4.2	Autoencoder	16
4.3	LSTM	16
4.4	Initial Training Failure and Root Cause Analysis	17
4.5	Post-Rebuild Results	17
4.6	Multi-Mechanism Convergence: Smurfing as Case Study	18
4.7	Framing for Production Deployment	18
5	Streaming Pipeline and Integration	20
5.1	Topology and Broker	20
5.2	Core Extraction Pattern	20
5.3	Dead Letter Queue and Fault Tolerance	20
5.4	Enrichment Service	21
5.5	Scoring Service	21
5.6	Decision Service	22
5.7	Sink Consumers and Persistence	22
5.7.1	Deliberate FK Omission	22
5.7.2	Schema Overview	22
5.8	Batch Integration	23
6	MLOps, Observability, and Production Readiness	24
6.1	MLOps Lifecycle Overview	24
6.2	Experiment Tracking with MLflow	24
6.3	Model Promotion Gate	24
6.4	Orchestration with Apache Airflow	25
6.5	Retraining Triggers	25
6.6	Supervisor Feedback Loop	26
6.7	Observability and Monitoring	26
7	Dashboard and Results	28
7.1	Dashboard Architecture	28
7.2	Pages	28
7.3	Pipeline Validation and Results	29

8	Production Roadmap	31
	General Conclusion	32
	Bibliography	33
A	Enriched Feature Schema	34

List of Figures

6.1	Drift-triggered retraining chain. The webhook relay exists because AlertManager cannot natively authenticate against Airflow's REST API.	25
-----	--	----

List of Tables

2.1	Common envelope fields	7
2.2	Client profile categories	8
2.3	Enriched event feature categories (30 features; full list in Appendix A)	8
2.4	Cold-start scoring progression	9
2.5	Regulatory escalation rules	10
3.1	Client archetypes	13
3.2	Anomaly scenarios by difficulty tier	14
3.3	Dataset validation	14
4.1	Data split strategy	16
4.2	Model performance comparison	17
4.3	Per-scenario detection performance (rebuilt models)	18
5.1	PostgreSQL tables and their roles	23
6.1	Retraining trigger comparison	26
7.1	Alert tier distribution on the synthetic dataset	29
7.2	Model AUC-ROC on the 30-feature schema	29
A.1	Complete enriched feature schema (30 features)	34
A.2	Docker Compose service inventory	35

List of Abbreviations

AE	Autoencoder	
AML	Anti-Money Laundering	
AUC	Area Under the Curve	
BCT	Banque Centrale de Tunisie	
CTAF	Commission Tunisienne des Analyses Financi	eres
DI	Dependency Injection	
DLQ	Dead Letter Queue	
DT	Dinar Tunisien	
E2E	End-to-End	
FK	Foreign Key	
FT	Financing of Terrorism	
KPI	Key Performance Indicator	
LAB	Lutte Anti-Blanchiment	
LSTM	Long Short-Term Memory	
MLOps	Machine Learning Operations	
NLG	Natural Language Generation	
ROC	Receiver Operating Characteristic	
SHAP	SHapley Additive exPlanations	

General Introduction

The digital transformation of retail banking has pushed the volume and velocity of financial transactions well beyond what manual oversight can monitor. In a network of 150 branches processing thousands of teller operations daily, the gap between transaction speed and supervisory capacity creates exposure to fraud, money laundering, operational errors, and behavioral anomalies that may go undetected until a periodic compliance review surfaces them — often weeks or months after the fact.

Amen Bank’s Direction Centrale du Système d’Information already operates an AI-based anti-money laundering system oriented toward bank-wide compliance reporting. This internship addresses a complementary need: **real-time decision support for branch supervisors at the point of transaction**. The objective is to design and build a platform that ingests teller operations as they occur, profiles both the client and the employee involved, detects anomalies using deep learning models, and surfaces explainable alerts that a non-technical supervisor can understand and act upon immediately.

The platform covers four teller-window operations — *retrait*, *versement*, *virement*, and *remise de chèques* — and monitors two independent actors per transaction: the client initiating it and the employee processing it. An Autoencoder detects point anomalies in individual transactions while an LSTM network identifies sequential deviations in client behavior over time. Every alert carries a human-readable explanation generated through SHAP, bridging the gap between model output and supervisory action.

Beyond the detection models themselves, the project invests in the infrastructure that makes a machine learning system operational and maintainable: a streaming pipeline built on Red-Panda, behavioral profiles maintained through a hot/cold data layer, experiment tracking with MLflow, three independent retraining triggers, structured logging, and Prometheus-based observability. The system runs as 20 Docker containers orchestrated through Docker Compose, validated end-to-end on a synthetic dataset of 243,000 transactions generated from five client archetypes calibrated against the bank’s published data.

This rapport is organized as follows. Chapter 1 presents the institutional and regulatory context and reviews the anomaly detection techniques that inform the design. Chapter 2 describes the architecture, the data schemas, and the key design decisions. Chapter 3 covers synthetic data generation. Chapter 4 details model training and evaluation. Chapter 5 presents the streaming pipeline and its integration with the persistence and batch layers. Chapter 6 covers the MLOps layer. Chapter 7 describes the supervisor dashboard and the end-to-end results. Chapter 8 documents the enhancements required for a production banking deployment.

1 Context and State of the Art

1.1 Host Institution

Amen Bank is a privately held Tunisian commercial bank operating under the supervision of the Banque Centrale de Tunisie (BCT). As of 2023, it operates 150 branches across 9 geographic zones, employing 1,139 staff of whom 60.5% are assigned to the branch network. Its client base is composed of individuals (61% of deposits), private businesses (33%), and institutional clients (6%) — a composition drawn from the audited annual report that directly calibrates the client archetypes of Chapter 3.

The internship was hosted by the **Direction Centrale du Système d'Information** (DCSI), within the *Département des Projets Agence*, which develops and maintains the information systems supporting branch-level operations.

1.2 Branch Teller Operations

The project scope covers the four teller-window (*guichet*) operations that constitute the core transactional activity of a retail branch:

- **Retrait** (cash withdrawal) — risk exposure includes identity fraud, unusual withdrawal patterns, excessive amounts or frequency.
- **Versement** (cash deposit) — the primary vector for *structuring* (smurfing): splitting a large sum into multiple deposits below the reporting threshold.
- **Virement** (bank transfer) — transfers to unknown beneficiaries, duplicates, amounts inconsistent with the client profile.
- **Remise de chèques** (cheque deposit) — unknown emitters, amounts inconsistent with history, third-party endorsement patterns.

Each operation involves **two actors**: the client initiating it and the employee processing it at the window. Anomalies can originate from either side — a client structuring deposits, or an employee processing an unusual volume of high-value transactions for a concentrated set of clients. This dual-actor nature motivates the independent client and employee profiling of Chapter 2.

1.3 Regulatory Framework

Tunisian banks operate under a layered anti-money laundering (LAB) and counter-terrorism financing (FT) framework involving three institutional actors.

BCT issues binding circulars defining internal control obligations, including the obligation to report transactions exceeding 10,000 DT to the financial intelligence unit. Circulars 2018-09, 2017-08, and 2021-05 govern LAB/FT internal controls but are restricted documents not publicly available. The system's regulatory rules are therefore calibrated from thresholds extractable from CTAF case studies and published legislation rather than a complete circular inventory — a gap documented explicitly rather than papered over.

CTAF (Commission Tunisienne des Analyses Financières) is Tunisia's financial intelligence unit. Its 2021 annual report provides the behavioral ground truth for what suspicious operations look like locally: *espèces* are involved in 68% of flagged cases, *virements* in 56%, *chèques* in 34%. The concentration of reported amounts around the 10,000 DT threshold — 85% of flagged operations in the 5,000–20,000 DT band — directly informed the near-threshold detection logic of the Decision Service.

Law n°41-2024, effective February 2025, caps individual cheques at 30,000 DT. BCT data from Q1 2026 confirms a 24.9% collapse in cheque usage volume following implementation. This

required adjusting the cheque archetype parameters and introduced a new anomaly scenario: cheque amounts clustering just below the ceiling, analogous to structuring below the reporting threshold.

1.4 Existing Detection Capabilities and Project Positioning

The Amen Bank 2023 annual report confirms that the bank already operates an AI-based AML system. Based on the report’s description, that system performs **bank-wide compliance reporting**: a compliance-officer audience, a quarterly tempo, and output aligned with CTAF regulatory requirements.

The system built in this internship occupies a different position under the same regulatory umbrella: **real-time decision support for branch supervisors** on four specific *caisse* operations, profiling client and employee as independent actors, at near-real-time latency, with sequence-aware LSTM modeling of behavioral drift. The two systems are complementary rather than competing — the existing one answers “*should the bank file a regulatory report?*” while this project answers “*should the branch supervisor investigate this transaction now?*”

1.5 State of the Art

1.5.1 Rule-Based and Statistical Detection

The earliest and still most widely deployed approach to transaction monitoring relies on deterministic rules: fixed thresholds, velocity checks, and pattern matching defined by compliance officers. Bolton and Hand [1] note that rule-based systems dominate production deployments due to their interpretability and auditability, but have two structural limitations: they require manual maintenance for every new fraud typology, and they cannot detect patterns outside the predefined rule set. Conversely, regulatory requirements often *mandate* deterministic checks (such as the 10,000 DT reporting threshold) that cannot be delegated to a probabilistic model. The system therefore retains deterministic rules in the Decision Service while delegating pattern discovery to the deep learning models.

Statistical methods model the distribution of normal behavior and flag significant deviations. **Isolation Forest** [2] builds an ensemble of random trees in which anomalies, being few and different, require fewer partitions to isolate. **Local Outlier Factor** [3] compares the local density of an observation to that of its neighbors. Both are efficient and distribution-free, and both are fundamentally *point-wise*: they evaluate observations independently and cannot model temporal dependencies. Detecting behavioral drift — a client whose pattern gradually shifts over weeks — requires models that consume sequences.

1.5.2 Autoencoders

An Autoencoder is a neural network trained to reconstruct its input through a compressed latent representation. Trained exclusively on normal data, it learns to encode and decode the patterns of legitimate transactions; anomalous inputs produce high reconstruction errors that serve as anomaly scores. Sakurada and Yairi [4] showed that Autoencoders outperform linear PCA-based methods when normal data occupies a nonlinear manifold, and An and Cho [5] formalized reconstruction error as an anomaly score.

In the financial domain this addresses the class imbalance problem directly: anomalies represent less than 1% of transactions, making supervised classification impractical without extensive labels. Training only on normal transactions removes the need for labeled anomalies entirely — a critical advantage where confirmed fraud labels are scarce and delayed by months. The system uses an Autoencoder as its **point anomaly detector**.

1.5.3 LSTM Networks

LSTM networks [6] address the vanishing gradient problem that prevents standard recurrent networks from learning long-range dependencies; their gating mechanism allows selective retention of information across long sequences. For anomaly detection the LSTM is trained as a *next-step predictor*: given a sequence of recent transactions it predicts the features of the next one, and the deviation between predicted and observed features becomes the anomaly score. Bontemps et al. [7] applied this to network intrusion detection, showing that it detects subtle sequential anomalies that point-wise methods miss.

In banking, the LSTM captures patterns that unfold over time: a client who normally transacts once a month suddenly executing five transactions in three days, or a gradual increase in transfer amounts to a new beneficiary. These are invisible to the Autoencoder, which evaluates each transaction in isolation. The system uses the LSTM as its **sequential anomaly detector**.

1.5.4 Score Fusion

When multiple detectors operate in parallel their scores must be combined. Aggarwal [8] surveys fusion strategies from simple averaging to learned combination; the key challenge is that different detectors have different score distributions, making naive averaging misleading. The system uses a *sigmoid-weighted* fusion that adapts to account maturity: new accounts favor the Autoencoder (which needs only one event) over the LSTM (which needs a populated sequence buffer), and the LSTM weight increases as the account matures. This dissolves the cold-start problem inherent to sequential models without a separate detection pathway.

1.5.5 Explainability with SHAP

Deploying a deep model in a regulated environment requires that every alert carry an explanation a non-technical supervisor can act on. SHAP [9] attributes a prediction to individual input features using game-theoretic Shapley values, with three properties that suit this context: local accuracy, missingness, and consistency.

For the Autoencoder the system uses **KernelSHAP**, a model-agnostic approximation treating the reconstruction-error function as a black box. DeepSHAP is inapplicable because the anomaly score subtracts the input from the reconstruction, breaking the forward-pass tracing DeepSHAP requires. For the LSTM, SHAP values over the full sequence-length-by-feature tensor are too granular for a supervisor to read; the system instead reports an *expected-versus-actual* comparison per interpretable feature, making deviations immediately legible.

1.5.6 Streaming Architectures

Real-time detection requires processing events as they arrive rather than in periodic batches. Kreps et al. [10] introduced the log-based streaming paradigm, where events are written to an append-only log partitioned by key and consumed by independent services at their own pace, decoupling producers from consumers and enabling horizontal scaling of individual stages. The system adopts this paradigm using RedPanda, a Kafka-compatible broker that eliminates the JVM and ZooKeeper dependencies while preserving API compatibility. Partitioning by client identifier ensures that all transactions for a given client are processed in order — a requirement of the LSTM’s sequential modeling.

1.6 Problem Statement and Objectives

Detection of anomalous teller operations at Amen Bank currently relies on manual rules and periodic compliance reviews. This cannot scale to the transaction volume of 150 branches, cannot adapt to evolving typologies without manual rule updates, and provides no real-time decision support to branch supervisors.

The objective of this internship is to design and implement a real-time anomaly detection platform that:

1. ingests teller transactions as they occur and processes them through a streaming pipeline at near-real-time latency;
2. profiles both the client and the employee as independent behavioral actors;
3. combines a point anomaly detector (Autoencoder) with a sequential one (LSTM) to capture both individual deviations and behavioral drift;
4. accompanies every alert with a human-readable SHAP explanation;
5. provides a supervisor dashboard for alert review, operational monitoring, and transaction simulation;
6. implements the MLOps infrastructure — experiment tracking, automated retraining, drift detection, observability — required to maintain the system beyond initial deployment.

2 System Architecture and Design

This chapter presents the architecture, the data schemas that define the contract between services, and the design decisions that shaped the platform. Every choice is grounded in the operational constraints of branch teller banking in the Tunisian regulatory context.

2.1 System Overview

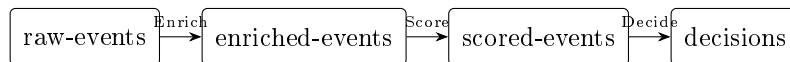
The platform follows a **streaming-first** architecture: four microservices chained via message topics, supported by a hot/cold data layer and a batch pipeline for profile rebuilding and model retraining. Each transaction passes through four stages — ingestion of the raw event, enrichment with behavioral context, scoring by two complementary models, and a decision layer merging the ML signal with deterministic regulatory rules. The output is a tiered alert with a human-readable explanation, consumed by a supervisor dashboard.

Three foundational choices define the architecture:

1. **RedPanda over Apache Kafka.** A Kafka-compatible API without the JVM runtime or ZooKeeper dependency, reducing operational complexity at a throughput — hundreds of events per minute from 150 branches — that does not require Kafka’s horizontal scaling. Critically, the architecture remains language-agnostic at topic boundaries: any service can be rewritten in a JVM language without changing the topic contracts.
2. **Dual-actor profiling.** Client and processing employee are independently profiled, each with a hot profile in Redis and a cold profile in PostgreSQL. This is motivated by CTAF case studies in which collusion between a client and a specific teller was the key pattern.
3. **SHAP co-located in the Scoring Service.** Explainability is computed at scoring time, not display time, and travels as part of the event payload. This reduces the dashboard to a single-topic consumer and guarantees that every alert carries its explanation regardless of how it is consumed downstream.

2.2 Streaming Topology

Four RedPanda topics form the backbone, all partitioned by `client_id`:



Partitioning by `client_id` is a deliberate constraint driven by the LSTM: it treats each client’s history as a temporal sequence, so cross-operation ordering must be preserved per client. A *retrait* followed by a *versement* followed by a *virement* must reach the Scoring Service in exactly that order, which partitioning by `client` guarantees.

For the same reason, a **single raw-events** topic carries all four operation types using a tagged-union envelope. Four separate topics — one per operation — were rejected: if *retrait* and *virement* events travel on separate topics there is no guarantee of relative ordering at the consumer.

A fifth topic, `d1q`, receives events that failed processing after retry exhaustion, preserving the original event alongside error metadata. Two sink consumers, on independent consumer groups, persist raw events and decisions to PostgreSQL, decoupling persistence from the transform-and-forward pipeline.

2.3 Hot/Cold Data Layer

Redis (hot) stores everything that must be available with sub-millisecond latency during enrichment: client and employee profiles, archetype baselines, per-client rolling event buffers for

velocity features, and LSTM sequence buffers. **PostgreSQL** (cold) is the source of truth: all transactions, alert history, profile snapshots for audit and retraining, and master reference data. It backs the batch pipeline and the dashboard’s queries.

The nightly batch pipeline reads PostgreSQL, recomputes aggregate statistics and behavioral distributions for both actors, writes snapshots back for audit, and refreshes the Redis hot profiles. The streaming services therefore never need to maintain complex windowed aggregations.

One subtlety: LSTM sequence buffers store *unscaled* feature vectors, with the Scoring Service applying the current scaler at inference time. A weekly retrain produces a new scaler; storing pre-scaled vectors would invalidate every persisted sequence and force a full buffer rebuild on each retrain.

2.4 Raw Event Schema

The raw event uses a **tagged-union envelope**: 11 common fields shared by all operations plus an operation-specific **payload**. Two principles govern it. First, raw events contain *only facts observable at transaction time* — no aggregate statistics, no derived fields; statistics belong in profiles and derived features in the enriched event. Second, *no lagging fields*: a cheque’s bounce status is only known after clearing and belongs to a separate downstream event.

Table 2.1: Common envelope fields

Field	Type	Description
<code>event_id</code>	UUID	Unique event identifier
<code>client_id</code>	UUID	Client initiating the transaction
<code>account_id</code>	UUID	Target account
<code>employee_id</code>	UUID	Teller processing the transaction
<code>branch_id</code>	String	Branch where the operation occurs
<code>timestamp</code>	Datetime	UTC timestamp
<code>amount</code>	Float	Transaction amount
<code>currency</code>	String	ISO 4217 code (typically TND)
<code>channel</code>	Enum	<code>guichet</code> , <code>GAB</code> , ...
<code>operation_type</code>	Enum	<code>retrait</code> , <code>versement</code> , <code>virement</code> , <code>cheque</code>
<code>payload</code>	JSON	Operation-specific data

A *virement* payload carries `beneficiary_id` and `motif`; a *cheque* payload carries `emitter_id`, `cheque_number`, and a `remise_id` grouping cheques deposited together. Each cheque generates its own event rather than one event per *remise*, preserving per-cheque granularity for scoring while keeping the grouping context.

2.5 Dual-Actor Profiling

The **client profile** comprises roughly 194 fields across six categories, summarized in Table 2.2.

Jensen–Shannon divergence between the 30-day and 180-day distributions captures behavioral drift in a single scalar per dimension. The risk history split by supervisor verdict enables both recidivist escalation and false-positive dampening in the Decision Service.

The **employee profile** mirrors this structure with two differences. Financial state is excluded — monitoring an employee’s personal finances is out of scope and ethically fraught. A *peer comparison* category is added: five z-scores at 30d and 90d covering volume, error rate, near-threshold ratio (fraction of transactions in the 8,000–9,999 DT band), client concentration, and client-location mismatch.

A normalization challenge follows from the branch structure: with only 2 tellers per branch, per-branch z-scores are statistically meaningless. The solution is **split normalization**. Volume

Table 2.2: Client profile categories

Category	Size	Content
Personal information	7	Archetype, maturity status, home branch, client type, account opening date
Transaction statistics	113	Count, sum, mean, std, min, max for each operation (4 × window (24h, 7d, 30d, 90d))
Behavioral patterns	12	Five frequency distributions (branch, hour, day-of-week, operation mix, employee) at 30d and 180d; known beneficiaries and emitters over 365d
Financial state	12	Balance statistics, inflow/outflow, net flow at 30d and 90d
Risk history	49	Alert counts per tier × window, each split into confirmed, rejected, pending
Sequence buffer	50 events	Bounded Redis list matching the LSTM training sequence length

metrics are first normalized per-branch (employee volume divided by branch average), then z-scored bank-wide — preventing high-volume branches from appearing as outliers while still detecting employees handling disproportionate volume within their branch. Ratio metrics are z-scored directly bank-wide, since ratios are already volume-independent.

2.6 Enriched Event Design

The enriched event is the feature vector consumed by the Scoring Service. Its redesign produced the single most impactful improvement in detection performance.

2.6.1 The Profile Dominance Problem

The initial schema contained 74 features, many of them raw profile field dumps — weekly snapshot means, standard deviations, and full distribution vectors repeated identically across every event from the same client within the same week. The per-event signal (how unusual *this* transaction is) was drowned out by profile noise (what the client’s long-term averages are). Both models failed to separate normal from anomalous events, producing AUC near random (~ 0.57).

2.6.2 Contrast Feature Redesign

The solution was to replace profile dumps with **contrast features**: each feature measures the deviation of the current event against the client’s established baseline. The count dropped from 74 to 30.

Table 2.3: Enriched event feature categories (30 features; full list in Appendix A)

Category	Count	Representative features
Raw event	7	amount, hour, operation one-hot, <code>is_home_branch</code>
Amount contrast	6	<code>z_amount</code> , ratio to mean, threshold and round-amount flags
Operation contrast	1	<code>op_type_probability</code> from the 30d distribution
Timing contrast	2	<code>is_outside_hours</code> , day distance from preferred day
Velocity	8	24h/7d counts, cumulative amount, duplicates, near-threshold count, employee counters
Context	6	archetype one-hot, <code>account_age_days</code>

The key invariant: the `_compute_features()` function producing these 30 features is a **pure function** — identical inputs always produce identical outputs, with no I/O and no side effects.

Both the streaming pipeline and the batch training pipeline call this same function, guaranteeing zero feature drift between training and inference. This is the single most important technical invariant in the system.

2.7 Scoring Architecture

2.7.1 Dual-Model Rationale

The **Autoencoder**, trained on normal transactions only, flags events with high reconstruction error. It excels at *point anomalies* — a 15,000 DT *retrait* from a salaried client averaging 300 DT — and works from the very first event, with no history required.

The **LSTM** predicts what a client’s next event should look like from their recent sequence. It excels at *sequential anomalies* — a gradual increase in *versement* frequency where each individual event looks normal but the trajectory signals structuring. It requires sufficient history to be meaningful.

2.7.2 Score Fusion

The two raw scores live on incompatible scales (reconstruction MSE versus prediction MSE). Three mechanisms reconcile them:

1. **Percentile normalization.** Each raw score is mapped to a percentile rank against a reference distribution computed from normal training data, placing both on a common $[0, 1]$ scale where 0.95 means “*higher than 95% of normal events.*”
2. **Maturity-adaptive weighting.** A sigmoid of `days_since_account_opening` (midpoint 90 days) controls the balance. This was chosen over event count, which penalizes low-frequency archetypes: a retiree transacting twice a month is not “new” but has a low event count.
3. **Weighted combination:** $S_{\text{fused}} = w_{\text{lstm}} \cdot S_{\text{lstm}} + (1 - w_{\text{lstm}}) \cdot S_{\text{ae}}$.

2.7.3 Cold-Start Scoring

Clients with fewer than 5 events in their sequence buffer fall outside the training distribution entirely, since the training pipeline applies a 21-day warm-up filter. A three-tier progression handles this:

Table 2.4: Cold-start scoring progression

Buffer length	Scoring behavior	Tier cap
0–4 events	Neutral score (0.5), <code>cold_start=True</code>	REVIEW max
5–9 events	AE only ($w_{\text{lstm}} = 0$)	Full range
10+ events	Full AE + LSTM fusion	Full range

Cold-start events still pass through all regulatory rules — a 15,000 DT transaction must trigger the BCT reporting flag regardless of buffer length — but the ML-derived tier is capped at REVIEW to prevent score saturation from generating false BLOCKs.

2.7.4 Conditional SHAP

SHAP is expensive (~ 100 ms per event via KernelExplainer) and roughly 95% of events score below the REVIEW threshold, so computation is gated behind the threshold check: below REVIEW the event takes the fast path (~ 5 ms). A deterministic template dictionary maps feature names to parameterized sentences, producing consistent and auditable explanations — “*This versement is suspect because the amount (8,450 DT) is $4.2\times$ the client’s average, and it is the 3rd deposit in 48 hours.*” An LLM-based generator was rejected on bank security grounds and for non-determinism.

2.8 Decision Service

The Decision Service is a stateless three-layer engine merging the ML signal with deterministic rules and historical context.

Layer 1 — score thresholds. Three thresholds calibrated on the validation set define the tiers: REVIEW at $T_1 = 0.95$ (review within 24 hours), ALERT at $T_2 = 0.99$ (immediate attention), BLOCK at $T_3 = 0.999$ (transaction held, requires override). Events below T_1 receive INFO: logged for audit, no action required.

Layer 2 — regulatory escalation. Eight rules can escalate the tier from Layer 1 but *never de-escalate* it. This is a hard invariant: a regulatory flag cannot reduce alert severity, and all flags are logged for audit whether or not they change the tier.

Table 2.5: Regulatory escalation rules

#	Rule	Condition	Escalation
1	BCT reporting	Amount $\geq 10,000$ DT and $z_{\text{amount}} \geq 2$	ALERT
2	Smurfing band	Amount in 8,000–9,999 DT and ≥ 2 near-threshold txns in 7d	ALERT
3	Cheque ceiling	Amount in 25,000–30,000 DT (Law n°41-2024)	REVIEW
4	Passeur de fonds	Occasional client, ≥ 5 versements in 24h	ALERT
5	New beneficiary	Virement $\geq 5,000$ DT to first-time beneficiary	REVIEW
6	Duplicate	Matching operation within time window	REVIEW
7	Round amount	Amount $\geq 5,000$ DT, round, high 24h cumulative	REVIEW
8	Frequency burst	$\geq 5 \times$ expected daily rate	REVIEW

Rule 1’s $z_{\text{amount}} \geq 2$ guard matters: a 15,000 DT *virement* from a big-business client averaging 20,000 DT is routine and should not flood the REVIEW queue. Only transactions that are *both* above the regulatory threshold *and* unusual for the specific client escalate.

Layer 3 — risk history adjustment. Clients with ≥ 3 confirmed alerts in 30 days receive a tier boost (a borderline event from a repeat offender is treated more seriously), while new accounts receive a more conservative posture reflecting the higher risk of unestablished baselines.

Explanation assembly depends on what triggered the alert: ML-only (SHAP contributions plus LSTM expected-vs-actual), rule-only (rule description with the specific values), or both (a combined explanation).

2.9 Software Architecture

A mid-project requirement — a synchronous REST endpoint for the dashboard’s simulation mode — exposed a coupling problem: all business logic lived inside Kafka consumer/producer wrappers, and a synchronous `/score` request cannot route through an asynchronous pipeline without unacceptable complexity. Request–reply over Kafka was rejected on latency; duplicating the logic outside Kafka was rejected on maintainability. The chosen option was an in-process orchestration layer, implemented through a **core extraction pattern**:

- **Core module** (`EnrichmentCore`, `ScoringCore`, `DecisionCore`): receives external dependencies — Redis client, model objects, metrics collector — via dependency injection, exposes a single public method (façade pattern), contains all business logic, and has *no* Kafka dependency.
- **Streaming wrapper**: thin consumer/producer plumbing that receives a core instance via DI; its poll loop calls the core’s public method and produces the result. Contains no business logic.

The FastAPI simulation API imports the three cores directly, wires them with shared Redis and model dependencies at startup via a lifespan context manager, and exposes a synchronous `/score` endpoint calling *enrich* $\rightarrow$ *score* $\rightarrow$ *decide* in-process. A `dry_run` flag on enrichment prevents simulation requests from polluting the streaming pipeline’s buffer state. The same pattern lets the batch pipeline call `_compute_features()` directly, guaranteeing feature parity across three execution contexts: streaming, batch, and simulation.

3 Synthetic Data Engineering

In the absence of real transaction data — a common constraint in banking anomaly detection projects — the quality of the synthetic dataset determines the ceiling of model performance. This chapter covers the source extraction methodology, archetype design, and the generator that produces labeled transactions.

3.1 Source Extraction Methodology

The generator requires two classes of calibration data: parameters describing the **normal population** (how legitimate clients transact) and patterns describing the **flagged population** (what suspicious activity looks like). No single source covers both, so a multi-source strategy was designed.

A structured extraction template scores each source across seven trust dimensions — recency, authority, peer review, information volume, geographic relevance, data type, and reproducibility — as Strong, Medium, or Weak. A nine-dimension coverage matrix then evaluates each source’s contribution: amount distribution, operation mix, frequency, inter-event time, temporal patterns, spatial/branch patterns, channel mix, counterparty known-rate, and *operation sequence patterns*. The last dimension was added specifically for the LSTM, which requires cross-operation temporal ordering.

3.2 Source Extractions

S1 — PaySim. A synthetic mobile-money simulator based on real transaction logs from an undisclosed African provider. Trust profile: 2 Strong (peer review, reproducibility), 4 Medium, 1 Weak (geographic relevance — mobile money in sub-Saharan Africa differs structurally from Tunisian branch banking). Its primary role is as a *beneficiary graph structure* source: the agent-to-agent transfer network is the best available model for counterparty relationship patterns. Secondary roles are amount distribution shape and day-of-month seasonality. Operation mapping is direct for three of four operations (CASH_OUT $\leftrightarrow$ retrait, CASH_IN $\leftrightarrow$ versement, TRANSFER $\leftrightarrow$ virement); cheque has no PaySim equivalent. The PaySim anomaly base rate of $\sim 0.13\%$ was retained as the initial injection-rate anchor.

S2 — Amen Bank 2023 Annual Report. Provides *normal population* calibration. The document has an internal two-tier trust structure: audited financial statements (signed by BDO Tunisie and La Générale d’Audit et Conseil) versus unaudited management narrative — a distinction that guided which figures were treated as hard constraints. Key extractions: the client deposit composition (61% individuals / 33% private business / 6% institutional) calibrating archetype proportions; 150 branches across 9 zones setting the branch pool; 1,139 employees with 60.5% network-assigned, which combined with the branch count yields the approximation of 2 tellers per branch; and confirmation of the existing AI-AML system, validating the project’s positioning as complementary.

S3 — CTAF 2021 Annual Report. Provides *flagged population* calibration. Four typological case studies were extracted with a structured 7-field template: espèces-heavy structuring below the 10,000 DT threshold dispersed across branches (informing the near-threshold ratio feature and the smurfing scenario); a virement chain to unknown beneficiaries with round amounts (informing first-time-beneficiary and round-amount rules); a politically exposed person used as a fund conduit (validating the decision to treat PPE as a structural flag layered on archetypes rather than a separate archetype); and third-party cheques deposited in rapid succession (informing cumulative amount tracking). Six quantitative targets were also extracted, including instrument co-occurrence rates and the concentration of BBE operations in the 5,000–20,000 DT band.

Known gap — BCT circulars. Circulars 2018-09, 2017-08, and 2021-05 would have provided an exhaustive threshold enumeration for internal LAB/FT controls, but are restricted documents. The gap is documented rather than papered over: the Decision Service’s rules derive from CTAF case studies and Law n°41-2024, not a complete circular inventory.

3.3 Client Archetype Design

Five archetypes were designed across six behavioral dimensions, calibrated from the S2 deposit composition.

Table 3.1: Client archetypes

Archetype	Population	Dominant op.	Monthly vol.	Amount range (DT)
Salaried	49%	Retrait (50%)	~5.7	200–500
Small Business	33%	Versement (55%)	~37.5	500–2,000
Student	5%	Retrait (95%)	~7.2	40–80
Retiree	7%	Retrait / Virement	~3.2	200–500
Big Business	6%	Virement (78%)	~63	2,000–20,000

Proportions derive from the 61/33/6 split, with the individual segment subdivided approximately 80% salaried, 12% retiree, 8% student. The six dimensions per archetype are operation mix, transaction frequency (mean and std per operation per month), timing (preferred day of month), amount ranges, counterparty loyalty, and branch loyalty. All parameters are externalized in a YAML configuration file, enabling recalibration without code changes.

A regulatory discovery during calibration — Law n°41-2024 capping cheques at 30,000 DT, followed by a 24.9% collapse in cheque volume — required revising the big-business archetype: *remise de chèques* frequency dropped from ~12–15 per month to 2–4, with *virement* absorbing the difference. A new anomaly scenario, cheque structuring just under the ceiling, was added.

3.4 Generator Architecture

The generator follows a four-class, two-phase design separating the behavioral model from the anomaly injection mechanism:

1. **Archetype** (layer 1) — loads behavioral parameters from YAML, defining population-level priors.
2. **Client** (layer 2) — samples a fixed personality from the archetype prior via Gaussian sampling with clipping. Each client’s parameters are drawn once at creation and remain fixed for life; operation mix probabilities are sampled then normalized to sum to 1.
3. **Generator** (orchestrator) — *Phase 1 (planning)* creates the population, estimates total event volume, and assigns anomaly scenarios to selected clients with specific time windows and difficulty tiers. *Phase 2 (generation)* ticks through each simulated day, determines which clients transact (bell curve around their preferred day), and emits normal or anomalous events depending on whether the client is inside an active anomaly window.
4. **Event** (inline) — per-event noise, payload construction, branch selection respecting branch loyalty, and employee assignment from the visited branch’s pool.

The two-phase approach is essential for controlled injection: hard-tier scenarios like smurfing span 21 days and require sufficient normal history *before* the window for the LSTM to have learned the baseline. Planning windows before generation guarantees this temporal constraint.

3.4.1 Branch-Linked Employee Assignment

A data quality requirement worth highlighting: each employee belongs to exactly one branch (2 per branch, 300 total). The initial implementation drew employees from a global pool, allowing

the same teller to appear at multiple branches — physically impossible in branch banking. This contaminated the `repeated_client_employee` scenario, which requires a specific client to visit a specific teller at a specific branch repeatedly and is meaningless if the teller can appear anywhere. After fixing the assignment, the LSTM’s AUC improved from 0.9430 to 0.9866, because branch-linked employees produce cleaner sequential patterns.

3.5 Anomaly Scenario Design

Nine scenarios span three difficulty tiers, where difficulty reflects how many features an anomaly modifies and over what horizon.

Table 3.2: Anomaly scenarios by difficulty tier

Tier	Scenario	Description	Actor
Easy (1d)	Amount spike	Single transaction $5\text{--}10\times$ client mean	Client
	Round amount	Large round amount (5,000–20,000 DT)	Client
	Duplicate virement	Matching virement within short window	Client
Medium (5d)	Frequency burst	Transaction rate $> 3\times$ daily average	Client
	Cumulative threshold	24h/48h cumulative exceeds norm	Client
	Timing anomaly	Transactions outside branch hours	Client
Hard (21d)	Smurfing	Virements in 8,000–9,800 DT, multiple branches	Client
	Cheque structuring	Cheques near the 30,000 DT ceiling	Client
	Repeated client-employee	Same client–teller pair ≥ 3 in 24h	Cross

Every scenario was validated for *feature-space detectability*: the injected anomaly must produce a measurable change in at least one of the 30 enriched features. Scenarios that modified only profile-level aggregates were removed during a mid-project audit, reducing the count from 13 to 9. The original `behavioral_drift` scenario, for example, gradually shifted a client’s operation mix over 21 days, but the shift was absorbed by the profile’s own 30-day distribution window and produced no detectable per-event signal.

The difficulty distribution follows a pyramid — 10% easy, 30% medium, 60% hard — reflecting the observation that most suspicious activity is designed to evade detection.

3.6 Data Validation

The generator produces four CSV files per run: `transactions.csv` (243,904 rows), `clients_master.csv` and `accounts.csv` (10,000 each), and `employees_master.csv` (300).

Table 3.3: Dataset validation

Metric	Value
Total events	243,904
Anomaly rate	0.623% (target $\sim 0.13\%$)
Anomaly scenarios	9 (all represented)
Unique clients	10,000 across 5 archetypes
Employees	300 (2 per branch, branch-linked)
Cross-branch employee violations	0
Simulation period	180 days (January–June 2025)
Salaried retraits mean	~ 300 DT (consistent with Tunisian salary)

The realized anomaly rate exceeds the 0.13% target because multi-day windows (5 days medium, 21 days hard) produce multiple anomalous events per plan entry rather than one.

This was accepted: the rate remains well within the imbalance range where reconstruction- and sequence-based models learn effectively, and AUC-ROC is threshold-independent.

3.6.1 Oracle Profile Shortcut

For training, client personality parameters are exported directly from the generator’s in-memory objects to `clients_master.csv`, providing the exact behavioral baseline needed for contrast features without a batch profile computation over simulated history. The shortcut produces the same 30 features as the production pipeline, preserving training–inference parity. In production these baselines are replaced by batch-computed profiles from real history — a substitution the pipeline handles without code changes.

4 Model Training and Evaluation

This chapter covers data preparation, the two model architectures, the diagnostic process that identified a fundamental feature engineering flaw, and the final performance after the pipeline rebuild.

4.1 Data Preparation

A **chronological** split preserves the temporal ordering the LSTM relies on; a random split would leak future events into training and inflate validation metrics.

Table 4.1: Data split strategy

Split	Day range	Events	Anomalies
Warm-up (discarded)	1–21	~28,000	—
Train (normal only)	22–126	142,254	0
Validation	127–153	36,548	312
Test	154–180	36,059	175

The first 21 days are discarded as a warm-up period during which client buffers are still populating and velocity features are unreliable. The Autoencoder training set contains **normal events only** — the standard approach for reconstruction-based detection: the model learns what normal looks like, and any event it cannot reconstruct well is flagged. A **StandardScaler** is fitted on the normal training data only, applied to all splits, and saved alongside the model for inference.

With an anomaly rate of 0.623%, accuracy is meaningless (predicting “normal” always yields 99.4%). AUC-ROC was chosen as the primary metric because it evaluates performance across all thresholds independently of class distribution, and serves as the Optuna objective function.

4.2 Autoencoder

The Autoencoder is a symmetric feedforward network:

$$30 \xrightarrow{\text{encoder}} 20 \longrightarrow 10 \xrightarrow{\text{decoder}} 20 \longrightarrow 30$$

A bottleneck of 10 (one-third of input) provides enough capacity to represent the structure of normal behavior without allowing the network to learn an identity function. ReLU activations, Adam optimizer (learning rate 10^{-3}), batch size 256, 50 epochs, best model selected by validation AUC.

The anomaly score is the mean squared reconstruction error between input x and reconstruction $\hat{x}$:

$$s_{\text{ae}} = \frac{1}{d} \sum_{i=1}^d (x_i - \hat{x}_i)^2$$

4.3 LSTM

The LSTM takes the last 10 enriched events and predicts the next:

$$\text{LSTM}(\text{input} = 30, \text{hidden} = 64, \text{layers} = 2) \rightarrow \text{Linear}(64, 30)$$

Two stacked layers process the sequence and the final hidden state feeds a linear projection predicting the 30 features of the next event. The sequence length of 10 balances behavioral

context against inference latency. The anomaly score is the mean squared prediction error between predicted $\hat{y}$ and observed y :

$$s_{\text{lstm}} = \frac{1}{d} \sum_{i=1}^d (y_i - \hat{y}_i)^2$$

Sequences are constructed per client with a sliding window: for each event at position t , the input is events $[t - 10, \dots, t - 1]$ and the target is event t . Clients with fewer than 11 events are excluded from LSTM training. In the streaming pipeline the sequence buffer lives in Redis and is updated after each event.

4.4 Initial Training Failure and Root Cause Analysis

The initial attempt used the 74-feature schema. Three Autoencoder configurations were evaluated via Optuna; the best achieved AUC 0.577. The LSTM with multi-task prediction heads achieved 0.564. Both were near random, triggering a root cause investigation that identified two independent causes:

1. **Undetectable scenarios.** Several of the original 13 scenarios had stub implementations producing no measurable change in feature space. `timing_anomaly`, for instance, shifted the event hour, but the enriched event contained no per-event hour feature — only a profile-level hour distribution that absorbs single-event changes.
2. **Profile dominance.** The 74-feature event was dominated by raw profile dumps identical across every event from the same client within the same week. Anomalous events looked nearly identical to normal events from the same client because both carried the same profile values.

Three components were redesigned simultaneously: the generator (13 scenarios reduced to 9, each validated for detectability), the enrichment layer (profile dumps replaced by 30 contrast features), and the training data (oracle profile shortcut, removing weekly granularity artifacts).

This was the single most impactful change in the project, and the improvement was entirely *structural*: the same architectures and hyperparameters that produced AUC 0.57 on the 74-feature schema produced AUC 0.98+ on the 30-feature contrast schema.

4.5 Post-Rebuild Results

Table 4.2: Model performance comparison

Model	Initial (74 feat.)	Rebuilt (30 feat.)	Final (aligned)
Autoencoder	0.577	0.9853	0.9714
LSTM	0.564	0.9866	0.9355

The *rebuilt* column reflects training on the oracle profile shortcut used during development; the *final* column reflects retraining on the production-aligned schema where streaming and batch call the same `_compute_features()` function. The drop between them was expected: the Autoencoder’s reconstruction error is less sensitive to input source differences, while the LSTM lost more because sequence modeling benefits from richer per-step representations.

An unexpected finding: hard-tier scenarios score *higher* than medium-tier ones. The inversion occurs because hard scenarios (smurfing, cheque structuring) modify high-signal features such as `z_amount` and `is_near_threshold` with large deviations, while medium scenarios produce subtler multi-event patterns requiring more context. The difficulty classification reflects real-world evasion effort, not model sensitivity.

Table 4.3: Per-scenario detection performance (rebuilt models)

Scenario	Tier	AE AUC	LSTM AUC
Amount spike	Easy	0.9998	0.9999
Round amount	Easy	1.0000	1.0000
Duplicate virement	Easy	0.9980	—
Frequency burst	Medium	0.9983	0.9125
Cumulative threshold	Medium	0.9742	0.8540
Timing anomaly	Medium	0.9710	—
Smurfing	Hard	1.0000	0.9981
Cheque structuring	Hard	1.0000	0.9999
Repeated client-employee	Hard	0.9983	0.9976

The two models show complementary strengths that justify the dual architecture. The Autoencoder excels at point anomalies visible in a single event and produces high scores from the very first one. The LSTM excels at sequential patterns; its weaker medium-tier performance reflects that those patterns span multiple events and demand sufficient context.

4.6 Multi-Mechanism Convergence: Smurfing as Case Study

Smurfing illustrates how the three detection mechanisms converge on one pattern. The scenario spans 21 days, yet neither the Autoencoder (one event at a time) nor the LSTM (a 10-event window) needs to observe the whole window.

Each smurfing event arrives already enriched with velocity features computed from the rolling Redis buffer: `near_threshold_count_7d` accumulates across the window, `cumulative_amount_24h` reflects recent activity, and `z_amount` captures the deviation from baseline. These features carry behavioral history *into* each event’s vector, regardless of the model’s own window size. The three mechanisms then contribute distinct signals:

1. **Autoencoder** — each event is anomalous in isolation: a salaried client averaging 300 DT suddenly making 8,500 DT versements produces high reconstruction error from `z_amount`, `amount_to_mean_ratio`, and `is_near_threshold` alone.
2. **LSTM** — the 10-event sequence shows escalating velocity features that deviate from the predicted trajectory; the model detects the acceleration within its own context without seeing the full 21 days.
3. **Decision rule 2** — fires deterministically when the amount falls in the 8,000–9,999 DT band with ≥ 2 near-threshold transactions in 7 days, requiring no ML inference at all.

This convergence is a direct consequence of the contrast feature design: because each enriched event carries both its own signal and accumulated behavioral context, even a stateless model participates in detecting a multi-day pattern. The LSTM adds temporal sensitivity; the regulatory rules provide a deterministic safety net.

4.7 Framing for Production Deployment

These AUC values validate the system’s ability to learn behavioral patterns and serve predictions end-to-end. They are *not* a production performance benchmark: the models were trained on synthetic data whose statistical properties approximate but do not replicate real Amen Bank distributions.

The engineering argument matters more than the absolute numbers. The system is designed so that replacing synthetic data with real transactions requires only re-running the training pipeline — the same feature function, the same fusion mechanism, the same decision rules, and the same retraining infrastructure handle real data without code changes. **The pipeline is the**

deliverable, not the model weights.

5 Streaming Pipeline and Integration

This chapter describes the runtime pipeline: how events flow from ingestion through enrichment, scoring, and decision in near real time, the software pattern that enables code reuse across streaming, batch, and API paths, the fault tolerance layer, and the integration with PostgreSQL and the nightly rebuild.

5.1 Topology and Broker

The pipeline is a linear chain of four topics, each consumed by a dedicated microservice producing to the next, all sharing `client_id` as partition key for the ordering reason established in Chapter 2.

RedPanda provides a Kafka-compatible API without the JVM or ZooKeeper dependency, which matters in a Docker Compose deployment where every service competes for memory: a Kafka cluster needs at least three containers, while RedPanda runs as one with the same producer/consumer API. The streaming wrappers use `kafka-python` and are unaware they are talking to RedPanda rather than Kafka — the switch is transparent at the application level.

Each service registers a dedicated consumer group, which provides two guarantees the pipeline relies on. *Offset tracking*: a restarted service resumes from its last committed offset rather than replaying the topic. *Horizontal scalability*: adding a second instance automatically splits partitions across the two consumers, doubling throughput without code changes. The current deployment runs one consumer per service, but the architecture supports scaling out by adding replicas.

5.2 Core Extraction Pattern

The system supports three execution paths that all require the same computation: streaming (Kafka loop, one event at a time, writes to Redis buffers), batch (Airflow DAG over PostgreSQL rows, no Redis), and the simulation API (synchronous request-response, read-only Redis). Duplicating enrichment, scoring, or decision logic across them would create both a maintenance burden and a drift risk.

Each stage therefore has a **core module** encapsulating all domain logic: `EnrichmentCore.enrich_event()`, `ScoringCore.score_event()`, and `DecisionCore.decide_event()`. Each core receives its dependencies through constructor injection — a Redis client, model objects, configuration — exposes a single public method, and has no knowledge of Kafka, Airflow, or FastAPI: it operates on Python dictionaries and returns Python dictionaries.

The Kafka-specific logic lives in thin wrappers that do exactly three things: poll a topic, call the core's public method, and produce the result. The `__main__` block is the sole wiring point — it creates the Redis client, loads models, constructs the core, injects it into the wrapper, and starts the loop.

The payoff was concrete. When the batch pipeline was rewritten it imported `EnrichmentCore` and called the same `_compute_features()` method; when the simulation API was built it imported all three cores and wired them with a shared Redis client. In both cases zero feature computation code was duplicated.

5.3 Dead Letter Queue and Fault Tolerance

In a streaming pipeline a single malformed event can block an entire consumer. The fault tolerance layer classifies failures into two categories with opposite responses.

Poison pills (`KeyError`, `ValueError`, `JSONDecodeError`) are structurally invalid and will never succeed regardless of retries; they are routed to the DLQ immediately with `retry_count=0`. **Tran-**

sient failures (`redis.ConnectionError`, `psycpg2.OperationalError`, network timeouts) may resolve on their own; the consumer applies exponential backoff (1 s, 2 s, 4 s) and retries up to three times before routing to the DLQ.

The distinction matters because the two classes demand opposite handling: retrying a poison pill wastes time and will never succeed, while immediately discarding a transient failure loses an event that would have succeeded one second later when Redis reconnected.

A `ResilientConsumer` class wraps all three streaming services. It intercepts exceptions from the core’s public method, classifies them with an `isinstance` check against a `TRANSIENT_ERRORS` tuple, applies the appropriate strategy, and produces failed events to a single `dlq` topic wrapped in a metadata envelope: the original event, source topic, failing service, error type and message, full stacktrace, retry count and maximum, and first/last failure timestamps. The envelope preserves enough context for an operator to diagnose the failure, replay the event after fixing the root cause, and distinguish infrastructure outages (transient, clustered in time) from data quality issues (poison pills, scattered across clients).

5.4 Enrichment Service

The Enrichment Service consumes raw events, computes the 30 contrast features by comparing the current event against the client’s historical profile, and produces enriched events.

For each event it fetches three items from Redis: the client profile (`profile:client:{id}`, batch-computed statistics — amount means and stds per operation, operation mix distributions, archetype, account age), the client’s rolling buffer of the last 50 raw event summaries for velocity features, and the employee buffer for peer comparison features. The profile is rebuilt nightly by the batch pipeline; the buffers are updated per event by the service itself after feature computation.

The term *contrast feature* is central: raw event fields carry no anomaly signal on their own, since a 10,000 DT versement is normal for a big business and extreme for a student. Every feature is computed relative to the client’s own baseline, turning absolute values into deviations comparable across archetypes.

Two implementation details deserve mention. When a client has no 30-day history for a given operation type — common for infrequent operations like cheque deposits — the z-score computation would divide by zero. A three-level **fallback chain** applies: 30-day statistics, then 90-day, then the archetype baseline, ensuring every event receives a meaningful contrast score regardless of transaction density. And after computing the features, the service appends the *unscaled* vector to `sequence:client:{id}`, capped at 10 entries; the scaler is owned by the Scoring Service, and storing pre-scaled features would couple the Enrichment Service to the scaler version, breaking independence during retraining.

5.5 Scoring Service

The Scoring Service runs both models and produces a fused score with optional SHAP explanations. The Autoencoder’s reconstruction error and the LSTM’s prediction error are each converted to a percentile rank against the training distribution, then combined with a sigmoid weight controlled by account maturity:

$$w_{\text{lstm}} = \sigma\left(\frac{d - 90}{15}\right), \quad \text{fused} = (1 - w_{\text{lstm}}) \cdot s_{\text{ae}} + w_{\text{lstm}} \cdot s_{\text{lstm}}$$

where d is days since account opening. For accounts younger than ~ 75 days the Autoencoder dominates; beyond ~ 105 days the LSTM does. The 90-day midpoint reflects the minimum history for the sequence model to produce reliable predictions.

Cold-start guard. Clients with fewer than 5 events in the sequence buffer pose a distinct

problem: their velocity features sit far outside the training distribution, which applied a 21-day warm-up filter. Without a guard, these zero-velocity vectors produce reconstruction errors mapping above the 0.99 percentile, generating a false BLOCK for every new client. The service therefore checks buffer length and returns a neutral 0.5 with $w_{\text{lstm}} = 0$ and no SHAP when the buffer holds fewer than 5 events, transitioning to AE-only at 5–9 and full fusion at 10+.

Conditional SHAP. Because KernelExplainer requires multiple model evaluations per event, explanations are generated only when the fused score exceeds $T_1 = 0.95$. Events below that threshold are overwhelmingly normal and their explanations would never reach a supervisor’s screen.

5.6 Decision Service

The Decision Service consumes scored events and produces the final tier — INFO, REVIEW, ALERT, or BLOCK — with an explanation combining model scores and rule triggers. The base tier comes from the three score thresholds of Chapter 2; the eight regulatory rules then evaluate, and each can *escalate* the tier but never de-escalate it.

The escalation-only principle exists because rules encode hard regulatory requirements that must never be overridden by a statistical model: a model might assign a low score to a transaction that happens to hit a reporting threshold, and the rule ensures the supervisor still sees it. Regulatory flags are logged for audit whether or not they change the tier.

Events flagged `cold_start=True` still pass through all rules — a genuinely suspicious transaction from a new client must trigger the appropriate flags — but the final tier is **capped at REVIEW**. This prevents the neutral 0.5 score from being escalated to ALERT or BLOCK by rules alone, which would generate high-priority alerts for clients who simply have not built enough history yet. The cap is recorded in the decision reasons so supervisors understand the limit.

5.7 Sink Consumers and Persistence

Two sink consumers persist streaming data to PostgreSQL. The `raw_events_sink` inserts every event into the `transactions` table, providing the permanent record used by the nightly profile rebuild and the dashboard’s transaction history. The `decisions_sink` performs two writes per event: an insert into the `alerts` table with tier, fused score, SHAP explanation as JSONB, and triggered rule names (with `supervisor_decision` initially NULL); and an increment of the client’s risk-history counters in Redis, which feed back into enrichment features for subsequent events.

5.7.1 Deliberate FK Omission

The `alerts` table has **no foreign key** to `transactions` despite sharing `event_id`. This is intentional. The two sinks run in parallel, and because the decision path adds latency (enrichment, scoring, rule evaluation), the decisions sink may attempt to insert an alert before the raw events sink has inserted the corresponding transaction. A foreign key would cause intermittent insertion failures depending on which consumer happens to be faster. The `event_id` still links the tables logically; the constraint is enforced at the application level rather than the database level.

5.7.2 Schema Overview

PostgreSQL is the cold store for the whole system, written by the two sinks and the batch pipeline, and read by the dashboard, the profile rebuild, and the retraining pipeline.

Three schema decisions are worth noting. The `tier` column uses a PostgreSQL `ENUM` rather than `VARCHAR`, enforcing valid values at the database level and enabling direct ordering (INFO < REVIEW < ALERT < BLOCK) without a `CASE` expression. Oracle labels live in a separate `evaluation_labels` table because they are generator metadata that does not exist in a production

Table 5.1: PostgreSQL tables and their roles

Table	Key	Role
<code>clients_master</code>	<code>client_id</code>	Archetype, account opening date, client type, home branch
<code>employees_master</code>	<code>employee_id</code>	Teller identity and assigned branch
<code>accounts</code>	<code>account_id</code>	Account reference data (FK to client)
<code>transactions</code>	<code>event_id</code>	All raw events; written by the raw events sink
<code>evaluation_labels</code>	<code>event_id (FK)</code>	Ground truth: oracle labels and supervisor verdicts
<code>alerts</code>	<code>alert_id</code>	Tier (ENUM), fused score, reasons, regulatory flags, JSONB explanation, supervisor decision
<code>profile_snapshots</code>	(client, date)	Nightly client profile as JSONB
<code>employee_profile_snapshots</code>	(employee, date)	Nightly employee profile as JSONB
<code>archetype_baselines</code>	<code>archetype</code>	Per-archetype reference statistics (final fallback)

system — separating them makes exclusion trivial without touching the transaction schema. And profiles are stored as **JSONB** rather than normalized columns: a client profile has ~194 fields across 6 categories, and normalizing them would produce a table whose shape changes whenever a feature is added. The tradeoff is that individual fields cannot be indexed efficiently, which the access pattern (load the whole profile into Redis) does not require.

5.8 Batch Integration

The streaming pipeline depends on client profiles too expensive to recompute per event. A nightly Airflow DAG (`nightly_profiles`) rebuilds them from the full transaction history through eight tasks with validation gates: rebuild client snapshots, validate completeness, rebuild employee snapshots, validate, recompute archetype baselines, validate, hydrate Redis, and run a smoke test scoring a sample event with the new profiles. Every task uses `ON CONFLICT` upserts on its target table, so any task can be retried after a mid-execution failure without duplicating or corrupting rows.

Feature parity guarantee. The batch computation calls the same `EnrichmentCore._compute_features()` method as the streaming path. Because the method is pure — taking event, profile, and buffer data as arguments and returning 30 features with no side effects — the only difference is that the batch script passes the archetype baseline as a parameter instead of fetching it from Redis. Any change to feature computation is therefore automatically reflected in both paths.

Streaming–batch coordination is a known limitation. The current deployment does not pause consumers during the rebuild window; in practice the DAG runs during low-traffic hours and the hydration step overwrites profile keys atomically, leaving a brief window in which consecutive events could read pre- and post-rebuild profiles. This one-event inconsistency is acceptable for the current deployment; a production system would pause consumers during hydration or swap profile versions inside a Redis transaction. This is documented as future work in Chapter 8.

6 MLOps, Observability, and Production Readiness

A trained model that cannot be retrained, versioned, or monitored is a liability rather than an asset. This chapter covers the infrastructure surrounding the models: experiment tracking, model promotion, retraining triggers, the supervisor feedback loop, and observability.

6.1 MLOps Lifecycle Overview

The MLOps layer turns the models of Chapter 4 from static artifacts into a managed lifecycle with four stages: **train** (produce a candidate with logged hyperparameters and metrics), **promote** (replace production only if performance is equal or better), **serve** (the streaming pipeline loads the promoted model from a shared volume mount, requiring no image rebuild), and **monitor** (observe the production score distribution and trigger a new cycle when drift or feedback indicates degradation).

The lifecycle is largely automated. The single human touch point is the supervisor confirming or rejecting alerts in the dashboard, which feeds one of the three retraining triggers. The promotion gate in particular has no manual approval step — a deliberate scope decision discussed below.

6.2 Experiment Tracking with MLflow

MLflow provides the tracking layer: every training job, whatever triggered it, logs to the same server, creating a single lineage of all model versions. The server runs as a Docker container with a flat-file backend (`mlruns/` mounted as a volume) rather than a database-backed store. For a single-user deployment running one job per week, this avoids the operational overhead of a dedicated metadata database; a production deployment serving concurrent training jobs would migrate to a PostgreSQL-backed store.

Each execution of `src/training/train_all.py` produces **two independent runs** under the experiment `amen-anomaly-detection`, one per model. They are independent rather than nested because the two models have different architectures, hyperparameters, and metrics — nesting would conflate unrelated configuration spaces. Each run logs parameters (architecture dimensions, learning rate, batch size, epochs, sequence length), metrics (validation AUC-ROC against the oracle labels), and artifacts (the trained `.pt` file, the fitted scaler, and the reference score distributions used by drift detection).

6.3 Model Promotion Gate

After training, the system must decide whether the candidate should replace the model currently serving. Each model type is evaluated independently: the script reads `models/production_metrics.json`, which stores the production AUC per type, and promotes the candidate if its AUC is $\geq$ the production value — writing its `.pt` file, scaler, and reference distribution to the shared model volume and updating the baseline.

The comparison uses $\geq$ rather than $>$ deliberately: a model retrained on fresher data with an identical AUC is preferable to one trained on older data, because the feature distributions it has seen are more representative of current traffic. On the very first run no baseline file exists; the script detects this and bootstraps by promoting unconditionally.

Two limitations are worth stating plainly. First, **no human approval step**: a model passing the AUC threshold is promoted immediately, whereas a production banking environment would require a compliance or risk officer to review model changes — an Airflow task pausing the DAG until confirmation is a straightforward extension. Second, **single-metric comparison**: the gate evaluates AUC only, where a more robust gate would add F1 at the operating threshold,

per-scenario recall to catch regressions on specific anomaly types, and inference latency to guard against deploying a model that is accurate but too slow for real-time scoring.

6.4 Orchestration with Apache Airflow

Airflow runs with a `LocalExecutor`, executing tasks in the scheduler process rather than distributing them — sufficient for two DAGs at low concurrency. Migrating to `CeleryExecutor` is a configuration change requiring no DAG modifications, only worker containers and a message broker. The Airflow metadata tables share the application PostgreSQL instance to conserve memory, and a dedicated `Dockerfile.airflow` installs only orchestration packages — excluding PyTorch, FastAPI, and SHAP — keeping the image roughly 2 GB smaller than the training image.

Beyond the nightly profile DAG of Chapter 5, the `weekly_retrain` DAG contains three tasks: *validate data freshness* (short-circuit if the `transactions` table has received no new events since the last run, avoiding wasted compute on an identical training set), *retrain models* via a `DockerOperator`, and *verify promotion* (confirm `production_metrics.json` was updated).

The `DockerOperator` was chosen over a Bash or Python operator for two reasons. *Dependency isolation*: training requires PyTorch, which is large and unnecessary in the scheduler. *Reproducibility*: the training image is pinned as `amen-anomaly-training:latest` with `force_pull=False`, so the same image serves scheduled, drift-triggered, and feedback-triggered retraining, and is rebuilt explicitly when training code changes rather than pulled on every run. The model volume is mounted into the training container at the same path used by the Scoring Service, so a promoted model is immediately available on the next restart — no copy or deployment step.

6.5 Retraining Triggers

Three independent paths can initiate retraining, and all converge on the same `weekly_retrain` DAG so that every job follows the same training, promotion, and artifact-logging pipeline.

Scheduled. A weekly cron in the DAG definition provides a baseline cadence, refreshing the models with recent data even when no anomaly is detected.

Drift-triggered. During training, the script saves the validation-set score distributions as reference arrays logged to MLflow. In production, the Scoring Service exposes the fraction of events scoring above T_1 as a Prometheus gauge, and an alert rule fires when that fraction exceeds 15% for a sustained hour:

```
alert: ScoreDistributionDrift    expr: score_above_t1_ratio > 0.15    for: 1h
```

The threshold and window were chosen empirically: under normal operation fewer than 5% of events exceed T_1 , so a sustained elevation above 15% indicates a material distribution shift rather than a single burst of legitimate anomalies. Figure 6.1 shows the resulting chain.

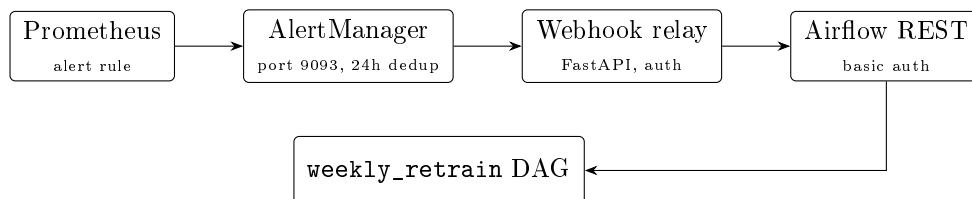


Figure 6.1: Drift-triggered retraining chain. The webhook relay exists because AlertManager cannot natively authenticate against Airflow’s REST API.

AlertManager deduplicates with a 24-hour `repeat_interval` to prevent repeated triggers

from one drift episode, then routes to a webhook receiver. That receiver — a single-endpoint FastAPI service on a minimal `python:3.11-slim` image (~150 MB) — exists purely because AlertManager’s native webhook format does not support the HTTP Basic Authentication that Airflow’s REST API requires. It performs the authentication and forwards the trigger with `conf={"trigger": "drift_alert"}`.

Feedback-triggered. Described in the next section.

6.6 Supervisor Feedback Loop

The feedback loop is the single human touch point in the lifecycle. When the pipeline produces an ALERT or BLOCK, the branch supervisor reviews it in the dashboard and either confirms or rejects it. The verdict is submitted to `POST /api/v1/feedback` on the FastAPI service, which writes it to the `evaluation_labels` table with a safe foreign key to the original transaction.

These labels serve two purposes. They provide ground truth for evaluating model performance on real decisions — a capability that becomes meaningful once the system runs on real transactions rather than synthetic events with oracle labels. And they feed a daily **feedback divergence DAG**, which queries what fraction of ALERT and BLOCK events were rejected over a rolling seven-day window and triggers `weekly_retrain` if the rejection rate exceeds 50% with at least 10 reviewed events.

The 50% threshold reflects a simple principle: if more than half of the system’s high-confidence alerts are being overridden by human reviewers, the models are producing more noise than signal at the upper tiers. The 10-review minimum prevents a single rejected alert early in a deployment from triggering an unnecessary cycle.

Table 6.1: Retraining trigger comparison

Trigger	Source	Condition	Frequency
Scheduled	Airflow cron	Weekly	Fixed
Drift	Prometheus alert	>15% above T_1 for 1h	On demand
Feedback	Supervisor reviews	>50% rejection, ≥ 10 reviews	Daily check

The `conf` parameter passed at trigger time records which path initiated a run, giving traceability in the Airflow logs without separate DAG definitions per trigger type.

6.7 Observability and Monitoring

Each streaming service embeds a `ServiceMetrics` class exposing two Prometheus instruments: a **Counter** of events processed, labeled by service and outcome, and a **Histogram** of per-event processing latency enabling percentile queries in Grafana. Services expose metrics on dedicated ports (enrichment 9100, scoring 9101, decision 9102), and a Kafka exporter (9308) provides broker-level metrics including *consumer group lag* — the gap between the latest produced and latest committed offset — which is the primary indicator of pipeline backpressure.

Failure domain separation. A monitoring system must not share a failure domain with the service it monitors. If the enrichment service and its health reporter both depended on Redis, a Redis outage would simultaneously break the service and suppress the alert reporting the breakage. This ruled out Redis-based heartbeats early in the design. The chosen architecture has Prometheus pull metrics over HTTP from each service’s embedded endpoint; the only shared dependency is the network layer itself, an acceptable residual risk since no monitoring architecture can report a network outage from inside the affected network.

Structured logging. A `JsonFormatter` class emits one JSON object per log line — timestamp, service, level, message, plus contextual extras such as `event_id`, `client_id`, `error_type`,

and `alert_tier`. A `setup_logger(service_name)` factory configures the standard logging module with this formatter, replacing all `print()` statements from earlier versions. Structured logs enable filtering by service, grouping by error type, or correlating events across services by `event_id` without fragile regular-expression parsing of free text.

7 Dashboard and Results

7.1 Dashboard Architecture

The Streamlit dashboard is the only component the branch supervisor interacts with directly; every other service is invisible infrastructure. It must therefore surface the right information clearly while remaining responsive under continuous use.

Why the dashboard cannot consume Kafka directly. Streamlit re-runs the entire script from top to bottom on every user interaction — clicking a button, changing a filter, navigating pages. A Kafka consumer embedded in the script would be destroyed and recreated on each interaction, losing its consumer group offset and replaying from the last committed position. Direct consumption is therefore architecturally incompatible with Streamlit. The solution is the sink consumer pattern of Chapter 5: two independent consumers write continuously to PostgreSQL and Redis, and the dashboard reads from PostgreSQL, which gives stable, queryable, filterable access to all historical alerts.

Data access. Reads go directly to PostgreSQL via `psycopg2` with Streamlit’s `@st.cache_data` decorator for query-level caching, with TTLs tuned per page: 10 seconds for the alerts list (freshness matters) and 30 seconds for KPI aggregations (stability matters more). A `safe_query()` wrapper handles connection resilience — if a query fails on a stale connection, common after Docker restarts or idle timeouts, it clears the cached connection, reconnects, and retries transparently. This single-retry pattern handles the most common failure mode without the complexity of a connection pool.

Writes go *exclusively* through FastAPI endpoints; the dashboard never issues `INSERT` or `UPDATE` statements. This preserves the single-writer principle and keeps all write-path logic — validation, audit logging, the feedback loop — in one place.

Refresh and navigation. The `streamlit-autorefresh` component triggers re-execution every 10 seconds. Streamlit supports no native server-to-client push, so periodic polling is the pragmatic alternative; 10 seconds balances responsiveness (a new `BLOCK` appears within 10 seconds of being written) against unnecessary database load. The dashboard uses Streamlit’s multi-page structure, so navigating executes only that page’s script, and a shared `components.py` module renders the sidebar with pipeline status and pending alert counts by tier.

7.2 Pages

Alerts — the supervisor’s primary workspace. Alerts are filterable by tier, operation type, and review status, each row showing a color-coded tier badge, operation, amount, timestamp, fused score, and status. Expanding an alert reveals the full context for a decision: client, employee, and branch identifiers; triggered rules with descriptions; regulatory flags with warning icons; and the SHAP explanation.

The explanation is presented in *two layers*. The first is the human-readable narrative from the deterministic NLG templates — sentences such as “*This versement is flagged because the amount is 4.2 times the client’s 30-day average and it is the third deposit within 48 hours.*” The second, behind an expander, shows raw feature-level contributions: per-feature SHAP values for the Autoencoder and expected-versus-actual deviations for the LSTM. This serves both audiences — supervisors needing a quick justification and analysts wanting the model’s internal reasoning.

Each pending alert offers **Confirm** and **Reject** buttons with an optional comment. Either sends a POST to `/api/v1/feedback`, which updates the alert’s supervisor decision and timestamp and upserts a record into `evaluation_labels`. The page then clears its cache and re-renders immediately rather than waiting for the next autorefresh.

KPIs — four metric cards (total alerts and the count for each actionable tier) above four Plotly charts: tier distribution as a pie chart, alerts grouped by operation type, alerts per day over the simulation period, and the top 20 branches by alert count. All aggregations use a 30-second cache TTL.

Simulation — lets a supervisor construct a hypothetical transaction and score it without injecting it into the live pipeline. This serves two purposes: testing whether a suspicious transaction would alert before it is processed, and training supervisors to understand how the system evaluates different profiles. The form collects client, employee, amount, operation type, and conditional payload fields, validates them client-side, and POSTs to `/api/v1/score`, which runs the full pipeline synchronously — enrichment, both models, fusion, SHAP, and decision rules — returning tier, score, triggered rules, and the same two-layer explanation.

Health — queries the Prometheus HTTP API for six service status indicators (three streaming services, two sinks, the simulation API) using the `up` metric, plus three operational metrics derived from the instruments of Chapter 6: throughput from the Counter’s rate, p95 processing latency from the Histogram, and consumer lag per partition from the Kafka exporter. If Prometheus is unreachable the page degrades to a warning rather than crashing — a monitoring outage should not block access to the other pages.

7.3 Pipeline Validation and Results

With all components integrated, the system was validated end-to-end by replaying the full 243,000-event dataset through the streaming pipeline and observing the resulting alert distribution.

Table 7.1: Alert tier distribution on the synthetic dataset

Tier	Proportion	Interpretation
INFO	89%	Normal operations, no action required
REVIEW	11%	Mild deviations, logged for audit
ALERT	0.35%	Significant anomalies, supervisor review
BLOCK	<0.1%	High-confidence anomalies, immediate action

Scores span 0.015 (routine transactions by established clients) to 0.999 (anomalies matching multiple detection mechanisms). The distribution is heavily right-skewed, with the vast majority of events well below T_1 — the expected shape for a functioning detector, where the tiers form a steep pyramid with very few events at the top.

Table 7.2: Model AUC-ROC on the 30-feature schema

Model	AUC (74 features)	AUC (30 features)
Autoencoder	0.9853	0.9714
LSTM	0.9866	0.9355

The decrease after feature reduction is expected and desirable. The 74-feature schema included raw profile statistics that made some anomalies trivially separable — a client whose `tx_count_30d` is zero but who suddenly transacts is obvious to any model. The 30-feature contrast schema forces detection from *relative* deviations rather than absolute values, which is harder but generalizes: the same features work across archetypes without archetype-specific thresholds. The LSTM loses more than the Autoencoder because it consumes sequences of 30-dimensional vectors, so each step carries less information per dimension, reducing its ability to model fine-grained temporal patterns. Both models nonetheless remain well above the 0.90 mark indicating

reliable discrimination.

Synthetic data caveat. These results validate the system’s ability to learn patterns, serve predictions at streaming latency, and produce a realistic alert distribution — but they are not a production benchmark. The generator produces anomalies from known scenarios with known parameters, while real anomalies are messier, more diverse, and not drawn from the same distribution. The engineering argument is different from the statistical one: plugging in real data requires no code changes. The generator is replaced by real event sources, the batch pipeline rebuilds profiles from real history, the models are retrained on real features from the same `_compute_features()` function, and the retraining triggers ensure the models adapt as the distribution evolves. The AUCs confirm that this pipeline works end-to-end, not that it will reproduce these numbers on production data.

8 Production Roadmap

This chapter documents the enhancements required to move the platform from its current Docker Compose deployment to a production banking environment.

Container orchestration. The system currently runs 20 Docker Compose services on a single host — sufficient for development and demonstration, but production requires horizontal scaling, automatic failover, and resource-aware scheduling across nodes. Kubernetes would replace Compose: each streaming service becomes a Deployment with a configurable replica count, and the Prometheus metrics endpoints already exposed can serve as liveness and readiness probes with no code changes. The model volume would migrate to a Persistent Volume Claim, preserving the retrain-without-rebuild pattern of Chapter 6. RedPanda, Redis, and PostgreSQL would run as StatefulSets with persistent storage, or be replaced by managed equivalents depending on the bank’s infrastructure policy.

CI/CD. Code changes are currently tested manually and deployed by rebuilding images. A production pipeline would add three stages: *continuous integration* running unit tests for the three core modules on every commit, validating that feature computation produces the expected 30-dimensional output, and linting; *image build* on merge to main, tagging images with the commit hash for traceability between deployed code and source control; and *staged deployment* to a staging environment first, running the E2E test script there and promoting to production only after validation. The core extraction pattern simplifies this considerably — the pure cores can be unit-tested without Redis, PostgreSQL, or RedPanda, covering most business logic with fast isolated tests.

Secrets management. Database credentials, API keys, and the Airflow REST password currently live in a `.env` file excluded from version control. This is adequate for a single developer but insufficient for production, where secrets must be rotated, audited, and never stored in plaintext on disk. A solution such as HashiCorp Vault or a cloud-native equivalent would provide encrypted storage, automatic rotation, and access logging, with Kubernetes Secrets injecting credentials at runtime and eliminating environment files.

Load testing. The system has been validated on 243,000 events replayed at accelerated speed but not under sustained production-equivalent load. A load testing phase would establish each service’s throughput ceiling, identify the first bottleneck, and determine the point at which consumer lag begins to accumulate. The existing simulator can serve as the load generator with minimal modification — adjusting the inter-event delay to simulate peak branch hours across 150 branches — and the existing Prometheus instrumentation already captures the metrics needed to evaluate the results.

Data retention. The schema currently retains all transactions and alerts indefinitely. Regulatory requirements and storage constraints make a retention policy necessary: Tunisian banking regulations require records to be preserved for a minimum period, after which they may be archived or purged. A retention strategy would partition `transactions` and `alerts` by month using PostgreSQL’s native partitioning, enabling efficient archival of older partitions to compressed cold storage while keeping recent data queryable at full speed. The nightly profile rebuild would then be scoped to the retention window rather than the full history, keeping computation time bounded as the dataset grows.

General Conclusion

This internship set out to design and build a real-time anomaly detection platform for branch teller operations at Amen Bank, covering *retrait*, *versement*, *virement*, and *remise de chèques*. The result is a working end-to-end system that ingests transaction events, computes behavioral features, scores them with two complementary deep learning models, applies regulatory rules, and surfaces explainable alerts to branch supervisors through an interactive dashboard.

The platform monitors two independent actors — the client initiating the transaction and the employee processing it — reflecting the dual-actor nature of teller operations where anomalies can originate from either side. A streaming architecture on RedPanda chains four microservices through message topics, supported by a hot/cold data layer and a batch pipeline for nightly profile reconstruction. The Autoencoder detects point anomalies while the LSTM identifies sequential deviations, with a sigmoid-weighted fusion that adapts to account maturity. Every alert carries a human-readable SHAP explanation from deterministic templates, letting supervisors act on the system’s output without machine learning expertise.

Beyond the detection pipeline, the project invested heavily in the infrastructure that makes a machine learning system maintainable. MLflow tracks every experiment. An automated promotion gate ensures only models matching or exceeding current production performance are deployed. Three independent retraining triggers — a weekly schedule, a Prometheus-driven drift detector, and a supervisor feedback divergence monitor — close the loop between serving and improvement. Structured JSON logging, Prometheus metrics, and Grafana dashboards provide the observability needed to diagnose issues without inspecting individual events.

The system was validated on a synthetic dataset of 243,000 events generated from five archetypes calibrated against Amen Bank’s published data. The validation confirms that the pipeline works end-to-end and that the retraining subsystem can detect drift and update models autonomously. The synthetic data caveat is explicit: these results demonstrate engineering readiness, not production-grade detection accuracy. The architecture is designed so that replacing synthetic data with real transactions requires no code changes — the same feature computation, the same models, and the same retraining pipeline apply without modification.

Two lessons stand out. Technically, the most impactful decision in the project was not a model choice but a feature engineering one: replacing profile dumps with contrast features raised AUC from 0.57 to over 0.98 with no change to architecture or hyperparameters, and enforcing a single pure feature function across streaming, batch, and simulation eliminated an entire class of train-serve skew. More broadly, building a model turned out to be a small fraction of the effort — the majority of the engineering work lies in the infrastructure that keeps that model correct, observable, and improvable over time.

Bibliography

- [1] R. J. Bolton and D. J. Hand, “Statistical Fraud Detection: A Review,” *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pp. 413–422, 2008.
- [3] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying Density-Based Local Outliers,” in *Proc. ACM SIGMOD*, pp. 93–104, 2000.
- [4] M. Sakurada and T. Yairi, “Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction,” in *Proc. MLSDA Workshop*, pp. 4–11, 2014.
- [5] J. An and S. Cho, “Variational Autoencoder based Anomaly Detection using Reconstruction Probability,” SNU Data Mining Center, Special Lecture on IE, 2015.
- [6] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] L. Bontemps, J. McDermott, and N.-A. Le-Khac, “Collective Anomaly Detection based on Long Short-Term Memory Recurrent Neural Networks,” in *Proc. ICDF2C*, pp. 141–152, 2016.
- [8] C. C. Aggarwal, *Outlier Analysis*, 2nd ed. Springer, 2017.
- [9] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.
- [10] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A Distributed Messaging System for Log Processing,” in *Proc. NetDB Workshop*, 2011.

A Enriched Feature Schema

The 30 features below are computed by the Enrichment Service for every transaction event and constitute the input vector for both the Autoencoder and the LSTM.

Table A.1: Complete enriched feature schema (30 features)

#	Feature	Category	Description
1	amount	Raw event	Transaction amount in DT
2	hour	Raw event	Hour of day (0–23)
3	is_retrait	Raw event	One-hot: operation is retrait
4	is_versement	Raw event	One-hot: operation is versement
5	is_virement	Raw event	One-hot: operation is virement
6	is_cheque	Raw event	One-hot: operation is remise de chèques
7	is_home_branch	Raw event	1 if transaction at client’s home branch
8	z_amount	Amount contrast	$(x - \mu_{30d})/\sigma_{30d}$ for this operation type
9	amount_to_mean_ratio	Amount contrast	x/μ_{30d} for this operation type
10	is_above_threshold	Amount contrast	1 if amount $\geq 10,000$ DT
11	is_near_threshold	Amount contrast	1 if amount $\in [8,000, 9,999]$ DT
12	is_round_amount	Amount contrast	1 if amount mod 1,000 = 0
13	is_near_cheque_ceiling	Amount contrast	1 if cheque amount $\in [25,000, 29,999]$ DT
14	op_type_probability	Operation contrast	Client’s 30d frequency ratio for this operation
15	is_outside_hours	Timing contrast	1 if hour $\notin [8, 16]$
16	day_distance_from_preferred	Timing contrast	day of month – preferred day
17	is_new_counterparty	Counterparty contrast	1 if beneficiary/emitter not in known set
18	tx_count_24h	Velocity	Client transactions in last 24 hours
19	tx_count_7d	Velocity	Client transactions in last 7 days
20	cumulative_amount_24h	Velocity	Sum of client amounts in last 24 hours
21	has_duplicate_recent	Velocity	1 if same amount + operation in last 24h
22	near_threshold_count_7d	Velocity	Events in [8k, 10k) DT band in last 7 days
23	employee_tx_count_24h	Employee	Employee’s transaction count in last 24h
24	same_employee_client_count_24h	Employee	Times this employee served this client in 24h
25	is_salaried	Context	One-hot: archetype is salaried

#	Feature	Category	Description
26	is_small_business	Context	One-hot: archetype is small business
27	is_student	Context	One-hot: archetype is student
28	is_retiree	Context	One-hot: archetype is retiree
29	is_big_business	Context	One-hot: archetype is big business
30	account_age_days	Context	Days since account opening

Features 8–9 use a three-level fallback chain when statistics are unavailable for the current operation type: 30-day statistics, then 90-day, then the archetype baseline. Feature 13 applies only to cheque operations and is zero otherwise. Features 18–22 are computed from the Redis rolling buffer rather than the PostgreSQL profile, so they reflect the most recent events with sub-second latency.

B Docker Compose Service Inventory

This appendix lists all 20 Docker Compose services that constitute the deployed system.

Table B.1: Docker Compose service inventory

Service	Image / Build	Port	Role
<i>Infrastructure</i>			
redpanda	vectorized/redpanda	9092	Kafka-compatible message broker
redis	redis:7	6379	Hot profile store and sequence buffers
postgres	postgres:17	5432	Cold store, alerts, evaluation labels, Airflow metadata
<i>Streaming Pipeline</i>			
enrichment-service	Custom (Dockerfile.api)	9100	Enriches raw events with 30 contrast features
scoring-service	Custom (Dockerfile.api)	9101	AE + LSTM inference, score fusion, SHAP
decision-service	Custom (Dockerfile.api)	9102	Rule evaluation, tier assignment
raw-events-sink	Custom (Dockerfile.api)	—	Writes raw events to PostgreSQL
decisions-sink	Custom (Dockerfile.api)	—	Writes alerts to PostgreSQL and risk history to Redis
<i>API and Dashboard</i>			
simulation-api	Custom (Dockerfile.api)	8000	FastAPI: scoring, feedback, health endpoints
dashboard	Custom (Dockerfile.api)	8501	Streamlit supervisor dashboard
<i>Orchestration</i>			
airflow-webserver	Custom (Dockerfile.airflow)	8080	Airflow UI and REST API
airflow-scheduler	Custom (Dockerfile.airflow)	—	DAG scheduling (LocalExecutor)
<i>MLOps</i>			
mlflow	ghcr.io/mlflow/mlflow	5001	Experiment tracking, artifact store
webhook-relay	Custom (Dockerfile.webhook)	—	AlertManager → Airflow REST API bridge
<i>Monitoring</i>			
prometheus	prom/prometheus	9090	Metrics collection and alert rule evaluation

Continued on next page

Service	Image / Build	Port	Role
alertmanager	prom/alertmanager	9093	Alert deduplication and web-hook routing
grafana	grafana/grafana	3000	Metrics visualization dashboards
kafka-exporter	danielqsj/kafka-exporter	9308	Broker-level metrics (consumer lag)
<i>On-Demand (profile-gated)</i>			
hydrate-redis	Custom (Dockerfile.api)	—	Seeds Redis profiles from PostgreSQL at startup
simulator	Custom (Dockerfile.api)	—	Replays synthetic events into RedPanda

Services are organized into six functional groups. The three streaming pipeline services and two sink consumers form the real-time processing chain. Infrastructure services (RedPanda, Redis, PostgreSQL) are shared across all components. The on-demand services (**hydrate-redis** and **simulator**) run once at startup or on explicit invocation using Docker Compose profiles, and exit after completion.

All custom services share a common `Dockerfile.api` base image except the Airflow containers (`Dockerfile.airflow`, which excludes PyTorch to save 2 GB), the webhook relay (`Dockerfile.webhook`, a minimal `python:3.11-slim` image at approximately 150 MB), and the training job (launched by the DockerOperator using a separate `amen-anomaly-training:latest` image not listed here, as it runs transiently during retraining).